



SQL functions and subqueries: A functional friendship

As you've been learning, SQL functions are tools built into SQL to facilitate performing calculations. For example, you could use the `AVG()` function to calculate the average salary of employees in a table so management knows what to budget for next year. Another example might be using the `COUNT()` function to count the number of orders in a table to track daily order inventory.

A subquery, also called an inner or nested query, is a SQL query that is nested inside a larger query. Going back to the previous example, you could add a subquery to your average calculation to identify the names of employees who earn more or less than the average salary to include that information in performance reviews. Subqueries allow more complex questions to be answered in a single query, making data retrieval more efficient. In this reading, you will learn about SQL functions and how they might be used with subqueries.

How do SQL functions function?

SQL functions help make data aggregation possible. As a refresher, data aggregation is the process of gathering data from multiple sources in order to combine it into a single, summarized collection. Take a moment to review some of these functions to better understand how to run these queries:

- **HAVING**: The **HAVING** clause filters the results of a SQL query based on conditions applied after the grouping. Check out [W3School's HAVING overview](#) for a tutorial on this clause
- **CASE**: **CASE** provides conditional logic in SQL queries, similar to an 'if-else' structure in programming languages. The [W3School's CASE overview](#) explores the use of the **CASE** statement and how it works.
- **IF**: **IF** performs a simple conditional test and returns a value depending on the outcome. Review [W3School's IF overview](#) for a tutorial of the **IF** function and examples that you can practice with.
- **COUNT**: **COUNT** performs a simple conditional test and returns a value depending on the outcome. Though it seems simple, the **COUNT** function is just as important as all the rest. The [W3School's COUNT overview](#) provides a tutorial and examples.

Subqueries

Subqueries can make projects easier and more efficient by allowing complex operations to be performed in a single query, reducing the need for multiple trips to the database. Subqueries also make your code more readable and maintainable. Take the employee salary example mentioned before.:The original query was used to find the average employee salary. By adding a subquery, you can learn this plus identify employees who earn more than the average—all in a single query.

Usually, you will find subqueries nested in the **SELECT**, **FROM**, and/or **WHERE** clauses. There is no general syntax for subqueries, but the syntax for a basic subquery follows a similar pattern:

```
SELECT account_table.*
FROM (
    SELECT *
    FROM transaction.sf_model_feature_2014_01
    WHERE day_of_week = 'Friday'
) account_table
```

```
WHERE account_table.availability = 'YES'
```

Basically, there's another **SELECT** clause inside the first **SELECT** clause. The second **SELECT** clause marks the start of the subquery in this statement. There are many different ways in which you can use subqueries, but there are a few rules to follow:

- Subqueries must be enclosed within parentheses.
- A subquery can have one or more columns specified in the **SELECT** clause.
- Subqueries that return more than one row can only be used with multiple value operators, such as the **IN** operator which allows you to specify multiple values in a **WHERE** clause.
- A subquery can't be nested in a **SET** command. The **SET** command is used with **UPDATE** to specify which columns (and values) are to be updated in a table.

Additional resources

The following resources offer more guidance into subqueries and their usage:

- [SQL subqueries](#): This detailed introduction includes the definition of a subquery, its purpose in SQL, when and how to use it, and what the results will be.
- [Writing subqueries in SQL](#): Explore the basics of subqueries in this interactive tutorial, including examples and practice problems that you can work through.

As you continue to learn more about using SQL, functions, and subqueries, you will realize how much time you can truly save when memorizing these tips and tricks.

